MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: MOHIT SHARMA

Subject & Code: DMW & 5AD5-16

Branch: CSE 5TH SEM

Batch: AI&DS

Semester: Vth

Session: 2022-23(odd Sem.)

Unit No.: 4

Date of Submission: 27-12-2023

Lect.No.	Topics Covered	Page No.	Total pages
1	Frequent pattern mining	2-3	2
2	Frequent item set mining method	4-5	2
3	Apriori algorithm, FP Growth algorithm	6-11	6
4	Frequent item set using vertical data format	12-13	2
5	Mining closed and max patterns	14-16	3
6	Advanced pattern mining	17-18	2
7	Pattern mining in Multilevel, Multidimensional space	18-23	6

References Book:

1. **Jiawei Han and Micheline Kamber, Data Mining: Concepts and Techniques, Morgan Kaufmann Publishers, third edition ,2013**
2. **Pang-Ning Tan, Michael Steinbach, Anuj Karpatne, Vipin Kumar, Introduction to Data Mining, second edition, Pearson, 2019**
3. **Ian. H. Witten, Eibe Frank and Mark. A. Hall, Data Mining: Practical Machine Learning Tools and Techniques, third edition , 2017**

FACULTY HOD DEAN ACADEMICS

UNIT 4

FREQUENT PATTERN MINING IN DATA MINING

- pattern mining in data mining is the process of identifying patterns or associations within a dataset that occur frequently. This is typically done by analyzing large datasets to find items or sets of items that appear together frequently.
Frequent pattern extraction is an essential mission in data mining that intends to uncover repetitive patterns or itemsets in a granted dataset. It encompasses recognizing collections of components that occur together frequently in a transactional or relational database. This procedure can offer valuable perceptions into the connections and affiliations among diverse components or features within the data.
Here's an elaborate explanation of repeating arrangement prospecting:

- **Transactional and Relational Databases:**

Repeating arrangement prospecting can be applied to transactional databases, where each transaction consists of a collection of objects. For instance, in a retail dataset, each transaction may represent a customer's purchase with objects like loaf, dairy, and ovals. It can also be used with relational databases, where data is organized into multiple related tables. In this case, repeating arrangements can represent connections among different attributes or columns.

- **Support and Repeating Groupings:**

The support of a grouping is defined as the proportion of transactions in the database that contain that particular grouping. It represents the frequency or occurrence of the grouping in the dataset. Repeating groupings are collections of objects whose support is above a specified minimum support threshold. These groupings are considered interesting and are the primary focus of repeating arrangement prospecting.

- **Apriori Algorithm:**

The Apriori algorithm is one of the most well-known and widely used algorithms for repeating arrangement prospecting. It uses a breadth-first search strategy to discover repeating groupings efficiently. The algorithm works in multiple iterations. It starts by finding repeating individual objects by scanning the database once and counting the occurrence of each object. It then generates candidate groupings of size 2 by combining the repeating groupings of size 1. The support of these candidate groupings is calculated by scanning the database again. The process continues iteratively, generating candidate groupings of size k and calculating their support until no more repeating groupings can be found.

- **Support-based Pruning:**

During the Apriori algorithm's execution, aid-based pruning is used to reduce the search space and enhance efficiency. If an itemset is found to be rare (i.e., its aid is below the minimum aid threshold), then all its supersets are also assured to be rare. Therefore, these supersets are trimmed from further consideration. This trimming step significantly decreases the number of potential item sets that need to be evaluated in subsequent iterations.

- **Association Rule Mining:**

Frequent item sets can be further examined to discover association rules, which represent connections between different items. An association rule consists of an antecedent and a consequent (right-hand side), both of which are item sets. For instance, {milk, bread} => {eggs} is an association rule. Association rules are produced from frequent itemsets by considering different combinations of items and calculating measures such as aid, confidence, and lift. Aid measures the frequency of both the antecedent and the consequent appearing together, while confidence measures the conditional probability of the consequent given the antecedent. Lift indicates the strength of the association between the antecedent and the consequent, considering their individual aid.

- **Applications:**

Frequent pattern mining has various practical uses in different domains. Some examples include market basket analysis, customer behavior analysis, web mining, bioinformatics, and network traffic analysis. Market basket analysis involves analyzing customer purchase patterns to identify connections between items and enhance sales strategies. In bioinformatics, frequent pattern mining can be used to identify common patterns in DNA sequences, protein structures, or gene expressions, leading to insights in genetics and drug design. Web mining can employ frequent pattern mining to discover navigational patterns, user preferences, or collaborative filtering recommendations on the web.

Regular pattern extraction is a data extraction approach employed to spot repeating forms or itemsets in transactional or relational databases. It entails locating collections of objects that occur collectively often and possesses numerous uses in different fields. The Apriori algorithm is a well-liked technique utilized to effectively detect consistent itemsets, and association rule extraction can be carried out to obtain significant connections between objects.

There are several different algorithms used for frequent pattern mining, including:

1. Apriori algorithm: This is one of the most commonly used algorithms for frequent pattern mining. It uses a “bottom-up” approach to identify frequent itemsets and then generates association rules from those itemsets.
2. ECLAT algorithm: This algorithm uses a “depth-first search” approach to identify frequent itemsets. It is particularly efficient for datasets with a large number of items.
3. FP-growth algorithm: This algorithm uses a “compression” technique to find frequent patterns efficiently. It is particularly efficient for datasets with a large number of transactions.
4. Frequent pattern mining has many applications, such as Market Basket Analysis, Recommender Systems, Fraud Detection, and many more.

Advantages:

1. It can find useful information which is not visible in simple data browsing
2. It can find interesting association and correlation among data items

Disadvantages:

1. It can generate a large number of patterns
2. With high dimensionality, the number of patterns can be very large, making it difficult to interpret the results.

FREQUENT ITEM SET IN DATA SET (ASSOCIATION RULE MINING)

1. Frequent item sets, also known as association rules, are a fundamental concept in association rule mining, which is a technique used in data mining to discover relationships between items in a dataset. The goal of association rule mining is to identify relationships between items in a dataset that occur frequently together.

2. A frequent item set is a set of items that occur together frequently in a dataset. The frequency of an item set is measured by the support count, which is the number of transactions or records in the dataset that contain the item set. For example, if a dataset contains 100 transactions and the item set {milk, bread} appears in 20 of those transactions, the support count for {milk, bread} is 20.

3. Association rule mining algorithms, such as Apriori or FP-Growth, are used to find frequent item sets and generate association rules. These algorithms work by iteratively generating candidate item sets and pruning those that do not meet the minimum support threshold. Once the frequent item sets are found, association rules can be generated by using the concept of confidence, which is the ratio of the number of transactions that contain the item set and the number of transactions that contain the antecedent (left-hand side) of the rule.

4. Frequent item sets and association rules can be used for a variety of tasks such as market basket analysis, cross-selling and recommendation systems. However, it should be noted that association rule mining can generate a large number of rules, many of which may be irrelevant or uninteresting. Therefore, it is important to use appropriate measures such as lift and conviction to evaluate the interestingness of the generated rules.

Association Mining searches for frequent items in the data set. In frequent mining usually, interesting associations and correlations between item sets in transactional and relational databases are found. In short, Frequent Mining shows which items appear together in a transaction or relationship.

Need of Association Mining: Frequent mining is the generation of association rules from a Transactional Dataset. If there are 2 items X and Y purchased frequently then it's good to put them together in stores or provide some discount offer on one item on purchase of another item. This can really increase sales. For example, it is likely to find that if a customer buys **Milk** and **bread** he/she also buys **Butter**. So the association rule is **['milk]^['bread']=>['butter']**. So the seller can suggest the customer buy butter if he/she buys Milk and Bread.

Important Definitions :

- **Support :** It is one of the measures of interestingness. This tells about the usefulness and certainty of rules. **5% Support** means total 5% of transactions in the database follow the rule.

Support(A -> B) = Support_count(A ∪ B)

- **Confidence:** A confidence of 60% means that 60% of the customers who purchased a milk and bread also bought butter.

Confidence(A -> B) = Support_count(A ∪ B) / Support_count(A)

If a rule satisfies both minimum support and minimum confidence, it is a strong rule.

- **Support_count(X)**: Number of transactions in which X appears. If X is A **union** B then it is the number of transactions in which A and B both are present.
- **Maximal Itemset:** An itemset is maximal frequent if none of its supersets are frequent.
- **Closed Itemset:** An itemset is closed if none of its immediate supersets have same support count same as Itemset.
- **K- Itemset:** Itemset which contains K items is a K-itemset. So it can be said that an itemset is frequent if the corresponding support count is greater than the minimum support count.

Example On finding Frequent Itemsets – Consider the given dataset with given transactions.

TransactionId	Items
1	{A,C,D}
2	{B,C,D}
3	{A,B,C,D}
4	{B,D}
5	{A,B,C,D}

APRIORI ALGORITHM

Prerequisite – Frequent Item set in Data set (Association Rule Mining)
Apriori algorithm is given by R. Agrawal and R. Srikant in 1994 for finding frequent itemsets in a dataset for boolean association rule. Name of the algorithm is Apriori because it uses prior knowledge of frequent itemset properties. We apply an iterative approach or level-wise search where k-frequent itemsets are used to find k+1 itemsets.
To improve the efficiency of level-wise generation of frequent itemsets, an important property is used called *Apriori property* which helps by reducing the search space.

AprioriProperty –
All non-empty subset of frequent itemset must be frequent. The key concept of Apriori algorithm is its anti-monotonicity of support measure. Apriori assumes that
All subsets of a frequent itemset must be frequent(Apriori property).
If an itemset is infrequent, all its supersets will be infrequent.

Before we start understanding the algorithm, go through some definitions which are explained in my previous post.
Consider the following dataset and we will find frequent itemsets and generate association rules for them.

TID	items
T1	I1, I2 , I5
T2	I2,I4
T3	I2,I3
T4	I1,I2,I4
T5	I1,I3
T6	I2,I3
T7	I1,I3
T8	I1,I2,I3,I5
T9	I1,I2,I3

minimum support count is 2
minimum confidence is 60%
Step-1: K=1
(I) Create a table containing support count of each item present in dataset – Called **C1(candidate set)**

Itemset	sup_count
I1	6
I2	7
I3	6
I4	2
I5	2

(II) compare candidate set item's support count with minimum support count(here min_support=2 if support_count of candidate set items is less than min_support then remove those items). This gives us itemset L1.

Itemset	sup_count
I1	6
I2	7
I3	6
I4	2
I5	2

Step-2: K=2

- Generate candidate set C2 using L1 (this is called join step). Condition of joining L_{k-1} and L_{k-1} is that it should have (K-2) elements in common.
- Check all subsets of an itemset are frequent or not and if not frequent remove that itemset.(Example subset of{I1, I2} are {I1}, {I2} they are frequent.Check for each itemset)
- Now find support count of these itemsets by searching in dataset.

Itemset	sup_count
I1,I2	4
I1,I3	4
I1,I4	1
I1,I5	2
I2,I3	4
I2,I4	2
I2,I5	2
I3,I4	0
I3,I5	1
I4,I5	0

(II) compare candidate (C2) support count with minimum support count(here min_support=2 if support_count of candidate set item is less than min_support then remove those items) this gives us itemset L2.

Itemset	sup_count
I1,I2	4
I1,I3	4
I1,I5	2
I2,I3	4
I2,I4	2
I2,I5	2
I2,I5	2

Step-3:

- Generate candidate set C3 using L2 (join step). Condition of joining L_{k-1} and L_{k-1} is that it should have (K-2) elements in common. So here, for L2, first element should match.
  So itemset generated by joining L2 is {I1, I2, I3}{I1, I2, I5}{I1, I3, i5}{I2, I3, I4}{I2, I4, I5}{I2, I3, I5}

- Check if all subsets of these itemsets are frequent or not and if not, then remove that itemset.(Here subset of {I1, I2, I3} are {I1, I2},{I2, I3},{I1, I3} which are frequent. For {I2, I3, I4}, subset {I3, I4} is not frequent so remove it. Similarly check for every itemset)
- find support count of these remaining itemset by searching in dataset.

Itemset	sup_count
I1,I2,I3	2
I1,I2,I5	2

(II) Compare candidate (C3) support count with minimum support count(here min_support=2 if support_count of candidate set item is less than min_support then remove those items) this gives us itemset L3.

Itemset	sup_count
I1,I2,I3	2
I1,I2,I5	2

Step-4:

- Generate candidate set C4 using L3 (join step). Condition of joining L_{k-1} and L_{k-1} (K=4) is that, they should have (K-2) elements in common. So here, for L3, first 2 elements (items) should match.
- Check all subsets of these itemsets are frequent or not (Here itemset formed by joining L3 is {I1, I2, I3, I5} so its subset contains {I1, I3, I5}, which is not frequent). So no itemset in C4
- We stop here because no frequent itemsets are found further

---

Thus, we have discovered all the frequent item-sets. Now generation of strong association rule comes into picture. For that we need to calculate confidence of each rule.

Confidence –
A confidence of 60% means that 60% of the customers, who purchased milk and bread also bought butter.
Confidence(A->B)=Support_count(A∪B)/Support_count(A)
So here, by taking an example of any frequent itemset, we will show the rule generation.
Itemset {I1, I2, I3} //from L3
SO rules can be
[I1^I2]=>[I3] //confidence = sup(I1^I2^I3)/sup(I1^I2) = 2/4*100=50%
[I1^I3]=>[I2] //confidence = sup(I1^I2^I3)/sup(I1^I3) = 2/4*100=50%
[I2^I3]=>[I1] //confidence = sup(I1^I2^I3)/sup(I2^I3) = 2/4*100=50%
[I1]=>[I2^I3] //confidence = sup(I1^I2^I3)/sup(I1) = 2/6*100=33%
[I2]=>[I1^I3] //confidence = sup(I1^I2^I3)/sup(I2) = 2/7*100=28%
[I3]=>[I1^I2] //confidence = sup(I1^I2^I3)/sup(I3) = 2/6*100=33%

So if minimum confidence is 50%, then first 3 rules can be considered as strong association rules.

LimitationsofAprioriAlgorithm
Apriori Algorithm can be slow. The main limitation is time required to hold a vast number of candidate sets with much frequent itemsets, low minimum support or large itemsets i.e. it is

not an efficient approach for large number of datasets. For example, if there are 10^4 from frequent 1- itemsets, it need to generate more than 10^7 candidates into 2-length which in turn they will be tested and accumulate. Furthermore, to detect frequent pattern in size 100 i.e. v1, v2… v100, it have to generate 2^100 candidate itemsets that yield on costly and wasting of time of candidate generation. So, it will check for many sets from candidate itemsets, also it will scan database many times repeatedly for finding candidate itemsets. Apriori will be very low and inefficiency when memory capacity is limited with large number of transactions.

FP Growth Algorithm in Data Mining

In Data Mining, finding frequent patterns in large databases is very important and has been studied on a large scale in the past few years. Unfortunately, this task is computationally expensive, especially when many patterns exist.

The FP-Growth Algorithm proposed by ***Han in***. This is an efficient and scalable method for mining the complete set of frequent patterns by pattern fragment growth, using an extended prefix-tree structure for storing compressed and crucial information about frequent patterns named frequent-pattern tree (FP-tree). In his study, Han proved that his method outperforms other popular methods for mining frequent patterns, e.g. the Apriori Algorithm and the TreeProjection. In some later works, it was proved that FP-Growth performs better than other methods, including ***Eclat*** and ***Relim***. The popularity and efficiency of the FP-Growth Algorithm contribute to many studies that propose variations to improve its performance.

What is FP Growth Algorithm?

The FP-Growth Algorithm is an alternative way to find frequent item sets without using candidate generations, thus improving performance. For so much, it uses a divide-and-conquer strategy. The core of this method is the usage of a special data structure named frequent-pattern tree (FP-tree), which retains the item set association information.

This algorithm works as follows:

- First, it compresses the input database creating an FP-tree instance to represent frequent items.
- After this first step, it divides the compressed database into a set of conditional databases, each associated with one frequent pattern.
- Finally, each such database is mined separately.

Using this strategy, the FP-Growth reduces the search costs by recursively looking for short patterns and then concatenating them into the long frequent patterns.

In large databases, holding the FP tree in the main memory is impossible. A strategy to cope with this problem is to partition the database into a set of smaller databases (called projected databases) and then construct an FP-tree from each of these smaller databases.

FP-Tree

The frequent-pattern tree (FP-tree) is a compact data structure that stores quantitative information about frequent patterns in a database. Each transaction is read and then mapped onto a path in the FP-tree. This is done until all transactions have been read. Different transactions with common subsets allow the tree to remain compact because their paths overlap.

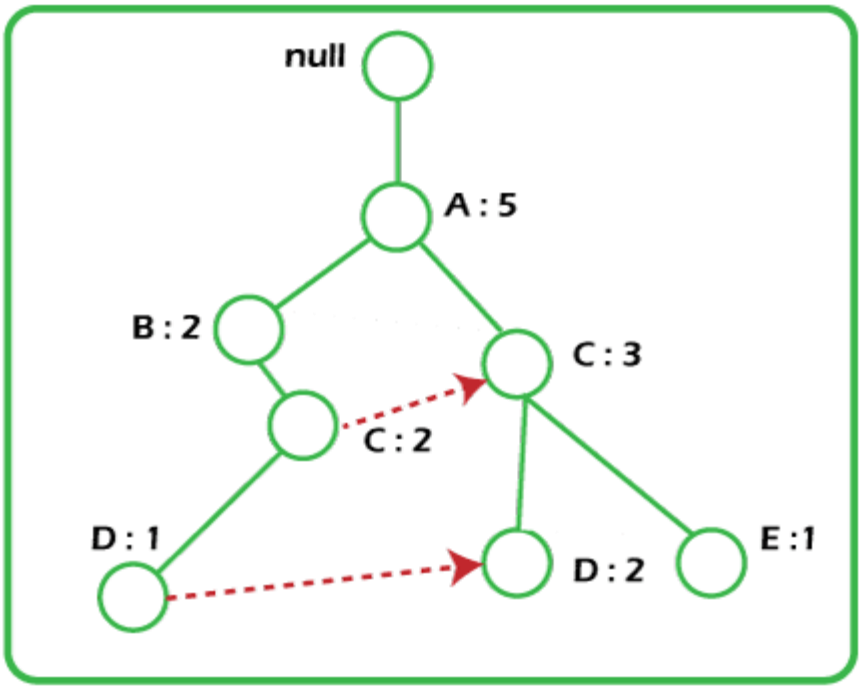
A frequent Pattern Tree is made with the initial item sets of the database. The purpose of the FP tree is to mine the most frequent pattern. Each node of the FP tree represents an item of the item set.

The root node represents null, while the lower nodes represent the item sets. The associations of the nodes with the lower nodes, that is, the item sets with the other item sets, are maintained while forming the tree.

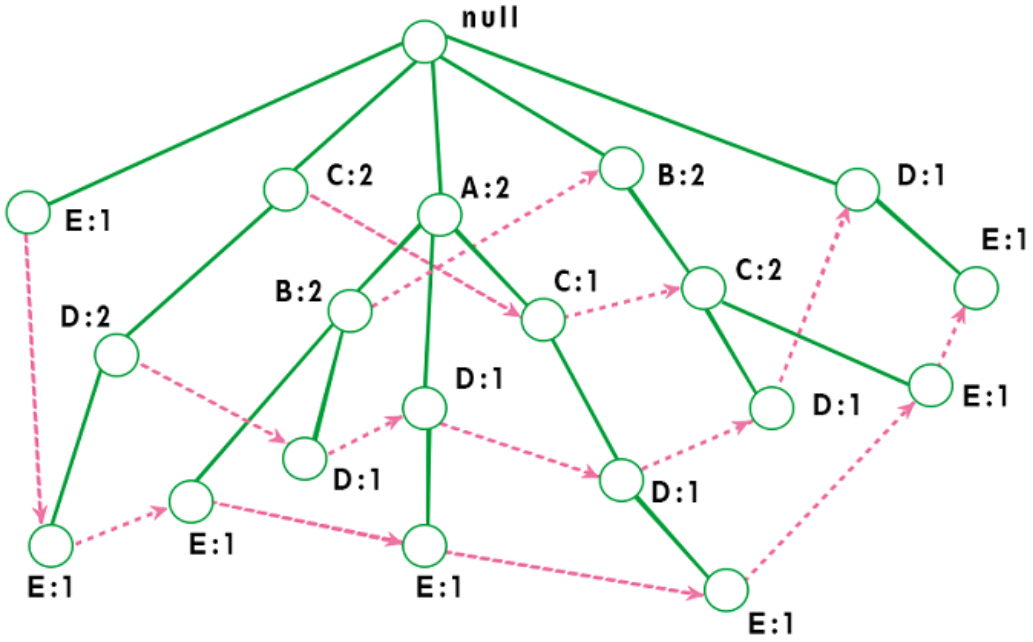
Han defines the FP-tree as the tree structure given below:

1. One root is labelled as "null" with a set of item-prefix subtrees as children and a frequent-item-header table.
2. Each node in the item-prefix subtree consists of three fields:
    - Item-name: registers which item is represented by the node;
    - Count: the number of transactions represented by the portion of the path reaching the node;
    - Node-link: links to the next node in the FP-tree carrying the same item name or null if there is none.
3. Each entry in the frequent-item-header table consists of two fields:
    - Item-name: as the same to the node;
    - Head of node-link: a pointer to the first node in the FP-tree carrying the item name.

Additionally, the frequent-item-header table can have the count support for an item. The below diagram is an example of a best-case scenario that occurs when all transactions have the same itemset; the size of the FP-tree will be only a single branch of nodes.



The worst-case scenario occurs when every transaction has a unique item set. So the space needed to store the tree is greater than the space used to store the original data set because the FP-tree requires additional space to store pointers between nodes and the counters for each item. The diagram below shows how a worst-case scenario FP-tree might appear. As you can see, the tree's complexity grows with each transaction's uniqueness.



MINING FREQUENT ITEMSETS USING VERTICAL DATA FORMAT

Introduction: Both the Apriori and FP-growth methods mine frequent patterns from a set of transactions in TID-itemset format (that is, {TID : itemsetg), where TID is a transaction-id and itemset is the set of items bought in transaction TID. This data format is known as horizontal data format. Alternatively, data can also be presented in item-TID set format (that is, fitem : TID setg), where item is an item name, and TID set is the set of transaction identifiers containing the item.

This format is known as vertical data format. In this section, we look at how frequent itemsets can also be mined efficiently using vertical data format, which is the essence of the ECLAT (Equivalence CLASS Transformation) algorithm developed by Zaki [Zak00].

Algorithm: FP growth. Mine frequent itemsets using an FP-tree by pattern fragment growth.

Input:

- D, a transaction database;
- min sup, the minimum support count threshold.

Output: The complete set of frequent patterns.

Method:

1. The FP-tree is constructed in the following steps:

(a) Scan the transaction database D once. Collect F, the set of frequent items, and their support counts. Sort F in support count descending order as L, the list of frequent items.

(b) Create the root of an FP-tree, and label it as "null." For each transaction Trans in D do the following. Select and sort the frequent items in Trans according to the order of L. Let the sorted frequent item list in Trans be [pjP], where p is the first element and P is the remaining list. Call insert tree([pjP], T), which is performed as follows. If T has a child N such that N.item-name=p.item-name, then increment N's count by 1; else create a new node N, and let its count be 1, its parent link be linked to T, and its node-link to the nodes with the same item-name via the node-link structure. If P is nonempty, call insert tree(P, N) recursively.

2. The FP-tree is mined by calling FP growth(FP tree, null), which is implemented as follows.

procedure FP growth(Tree,**α**)

(1) if Tree contains a single path P then

(2) for each combination (denoted as β) of the nodes in the path P

(3) generate pattern β∪**α** with support count = minimum support count o f nodes in β;

(4) else for each a_i in the header of Tree {

(5) generate pattern β = ai∪ a with support count = ai:support_count;

(6) construct β's conditional pattern base and then β's conditional FP tree Treeb;

(7) if Treeβ ≠0 then

(8) call FP growth(Treeβ, β); g

CLOSE AND MAX PATTERNS

Compress frequent patterns using closed patterns and max patterns

Transaction-id	Items bought
10	A, B, D
20	A, C, D
30	A, D, E
40	B, E, F
50	B, C, D, E, F

When we have many items, the total number of itemsets grow exponentially to the number of items in the database: |itemsets| = 2n. This immediately causes both space and time complexity problems in finding frequent patterns.

For example, the table above contains five transaction records and six items. This small database can have 26 = 64 possible itemsets. To deal with this problem, we now define a series of sets.

Sub-pattern (proper subset)

To find efficient algorithms for mining frequent patterns, we will start with two basic concepts of patterns. A pattern is an itemset, therefore the order of the items are not important. What will be useful is the properties of subsets of supersets of frequent patterns.

Let us start with subsets. We call any proper subset of a pattern, a sub-pattern. Here is an example,

- If a pattern X contains, 3 items: X={a, b, c}, any proper subsets, such as sets containing a, or b, or c, or ab, or ac, or bc, are all sub patterns.

Now, suppose we have a pattern with 100 items. How many sub-patterns can we find for this pattern?

- A lot, it is more than 10 to the power of 30.

That means if this pattern is a frequent pattern, there are 10 to the power of 30 frequent patterns.

Super-pattern (proper superset)

Another concept is of course super-pattern. A super pattern of a pattern is a proper super set of the pattern.

Here are some examples of the pattern containing the 3 items: a, b, c

- abcd, abce, abcde, …

Close Pattern

Now, we are ready for two important concepts of frequent patterns. Let's start with closed patterns.

A **closed pattern** is

1. a frequent pattern. So it meets the minimum support criteria.
2. In addition to that, all super-patterns of a closed pattern are less frequent than the closed pattern.

Formally, this is defined as:

CP is a **Closed Pattern** if
- CP is a **frequent pattern** and
- All super-patterns of CP are less frequent than CP
  - For all super-patterns S of CP,
    Supp(S) < Supp(CP)

Let's see some examples.

1. Suppose, the minimum support count is 2.
2. For the first example, suppose there are a total of 3 items: a, b, c.
3. Suppose a pattern ab has support count of 2 and a pattern abc has support count of 2.
4. Is the pattern ab is a closed pattern?

Pattern ab is a frequent pattern, but it has a super-pattern that is NOT less frequent than ab.

For the second example,

1. suppose there are a total of 3 items: x, y, z.
2. suppose a pattern xy has support count of 3 and a pattern xyz has support count of 2.
3. Is the pattern xy is a closed pattern?

Pattern xy is a frequent pattern and also the only super-pattern xyz is less frequent than xy.

Therefore, xy is a closed pattern.

Max Pattern

Next, let's define max pattern.

A **max pattern** is

1. a frequent pattern. So it also meets the minimum support criteria like closed pattern
2. In addition, but unlike closed pattern, all super-patterns of a max pattern are NOT frequent patterns.

This is formally written as

MP is a **Max Pattern** if
- MP is a frequent pattern and
- All super-patterns of MP are not frequent
  - For all super-pattern SP of MP,
    $Supp(SP) < Supp_{Min}$

Let's see some examples as well.

1. Suppose, the minimum support count is 2.
2. Like before, for the first example, suppose there are a total of 3 items: a, b, c.
3. Suppose a pattern ab has support count of 3 and a pattern abc has support count of 2.
4. Is the pattern ab is a max pattern?

Pattern ab is a frequent pattern, but it has a super-pattern that is a frequent pattern as well. So, pattern ab is NOT a max pattern.

For the second example,

1. suppose there are a total of 3 items: x, y, z.
2. Suppose a pattern xy has support count of 3 and a pattern xyz has support count of 1.
3. Is the pattern xy is a max pattern?

Pattern xy is a frequent pattern and also the only super-pattern xyz is NOT a frequent pattern. Therefore, xy is a max pattern.

ADVANCED PATTERN MINING

Due to extensive research, various extensions of the problem scope, and wide-ranging application studies, frequent pattern mining has advanced far beyond its foundational applications. You will discover techniques for sophisticated pattern mining in this chapter. We start by outlining a broad strategy for pattern mining. We address expanded uses of pattern mining and provide techniques for mining various types of patterns. We go into great detail about how to mine a variety of patterns, such as compressed and approximate patterns, multilayer patterns, multidimensional patterns, patterns in continuous data, rare patterns, negative patterns, constrained frequent patterns, and frequent patterns in high-dimensional data. This does not cover other pattern mining issues, such as mining sequential and organized patterns, mining patterns from spatiotemporal, multimedia, and stream data, as these are seen as more complex subjects. Observe how the term "pattern mining" is broader than "frequent pattern mining" because the former also refers to uncommon and unfavorable patterns. The two concepts, however, are synonymous when there is no ambiguity.

A Road Map

•**Basic patterns:** There are various different variations of a frequent pattern, such as a simple frequent pattern, a closed pattern, or a max-pattern. To recap, a common pattern is an item set (or pattern) that meets the minimum support requirement. If there isn't a superpattern p 0 with the same support as p, a pattern p is said to be closed. If there isn't acommon superpattern of pattern p, then pattern p is a max-pattern.

•In association rules or other types of rules based on interestingness metrics, frequent patterns can also be mapped. Sometimes, we may also be interested in uncommon or infrequent patterns (i.e., patterns that are uncommon but extremely significant, or negative patterns) (i.e., patterns that reveal a negative correlation between items).

•Depending on the types of values the rule or pattern handles: A rule is considered to be a Boolean association rule if it involves associations between items' existence or absence. for instance, are Boolean association rules discovered by market basket analysis. A rule is considered to be a quantitative association rule if it describes relationships between quantitative objects or qualities.

•Quantitative values for items or qualities are divided into intervals according to these guidelines. Rule (7.3), which discretizes the quantitative qualities age and income, can alternatively be thought of as a quantitative association rule. Depending on the restrictions or standards used to identify specific patterns: The patterns or rules that need to be found may be constraint-based (i.e., satisfying a set of user-defined constraints), approximate, compressed, near-match (i.e., those that add up the support counts of the nearly identical itemsets), top-k (i.e., the k most frequent itemsets for a user-specified value, k), redundancy-aware, or other types. top-k (i.e., the top-k patterns that do not contain redundant or similar patterns), and so on. Depending on the types of data and mining features: The majority of individuals are interested initemsets when given relational and data warehouse data. In this sense, frequent pattern mining is

WHAT IS MULTIDIMENSIONAL AND MUTLTILEVEL ASSOCIATION RULE IN DATA MINING?

This article delves into techniques for mining in a multi-layered, multi-dimensional space, utilizing both multilevel association rule in data mining and multidimensional association rule. It covers the discovery of unusual and negative patterns, as well as multilevel connections and multidimensional correlations. In the case of pattern mining multilevel relations, diverse levels of abstraction are linked together. At the same time, multidimensional relationships involve multiple dimensions or predicates (e.g., linking customers' purchase behavior to their age). Quantitative association rules can be created by assigning numbers as characteristics, with values with a predetermined hierarchy, such as age. Uncommon patterns refer to infrequent yet unique item combinations, while negative correlations between objects are also examined. Understanding multilevel association rules in data mining begins with understanding data science; you can get an insight into the same through our Data Science Training.

Association Rule in Data Mining

Strong correlations can be found at high levels of abstraction, even with great support; nonetheless, for many applications, these correlations can be regarded as common sense. There is a possibility that we will need to delve much farther into the data to recognize patterns that are genuinely one of a kind.

On the other hand, there may be many patterns at elementary abstraction levels, many of which are trivializations of patterns at higher degrees of abstraction. Consequently, it is fascinating to contemplate how to construct effective algorithms for mining patterns across a wide range of abstraction levels while ensuring that these algorithms have adequate flexibility to permit easy travel between these areas.

Example of Association Rule in Data Mining

You can make data more generalizable by replacing certain instances of low-level concepts with their higher-level concept relatives, also known as ancestors, from a concept hierarchy. This will make the data more generic.

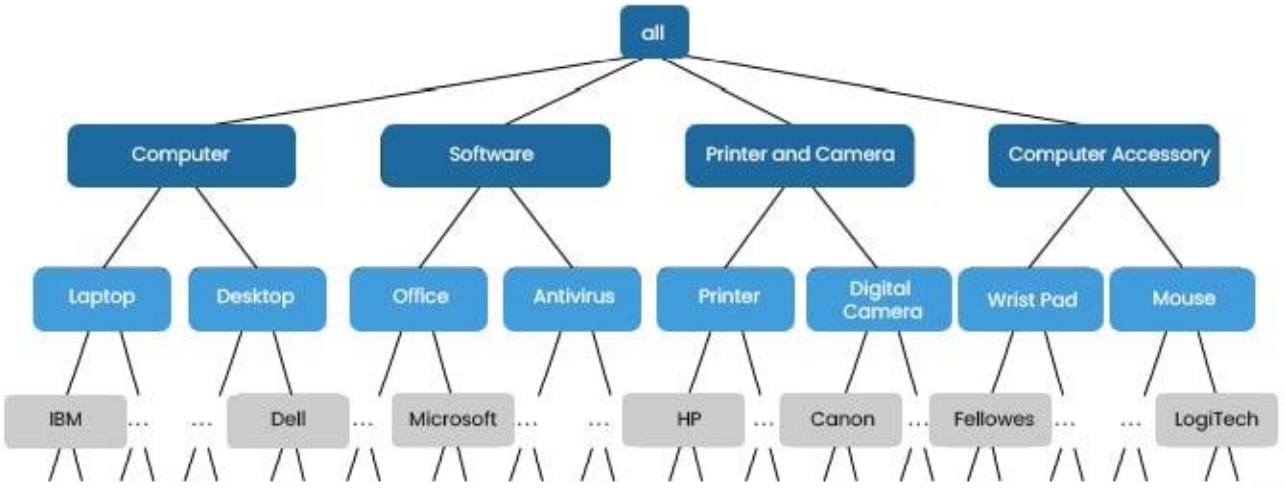
Example:

Let's imagine we have access to the critically crucial collection of sales data from an electronics store, displayed in Table 1. The ideas located at the bottom of an order are the ones that have the least generalization, while the concepts situated at the very top are the ones that are the most particular.

Task-Relevant Data, D

TID	Items Purchased
T100	Apple 17" MacBook Pro Notebook, HP Photosmart Pro b9180
T200	Microsoft Office Professional 2010, Microsoft Wireless Optical Mouse 5000
T300	Logitech VX Nano Cordless Laser Mouse, Fellowes GEL Wrist Rest
T400	Dell Studio XPS 16 Notebook, Canon PowerShot SD1400
T500	Lenovo ThinkPad X200 Tablet PC, Symantec Norton Antivirus 2010

In the order of five levels of concept, representation depicted, "Degree 0" stands for the most fundamental degree of abstraction. The first three levels are considered intermediate degrees of abstraction, while level 4 is considered the most significant level of abstraction. In this scenario, level 1 includes items such as computers, software, printers, cameras, and other computer accessories; level 2 includes laptops and desktops, as well as office and antivirus software; and level 3, includes high-end brands such as Dell desktops, Microsoft office software, and other similar products. Considering this hierarchy, the level 4 abstraction represents the most feasible detailed level. The numbers that are contained in the raw data have been incorporated.



Nominal attribute concept hierarchies are typically already present in the database schema. They can be automatically constructed with techniques such as those outlined. Data on product requirements were used to create the concept hierarchy for our example. Many discretization techniques were discussed to develop concept hierarchies for numeric characteristics. Users familiar with the data, like store managers in our example, may also provide idea hierarchies.

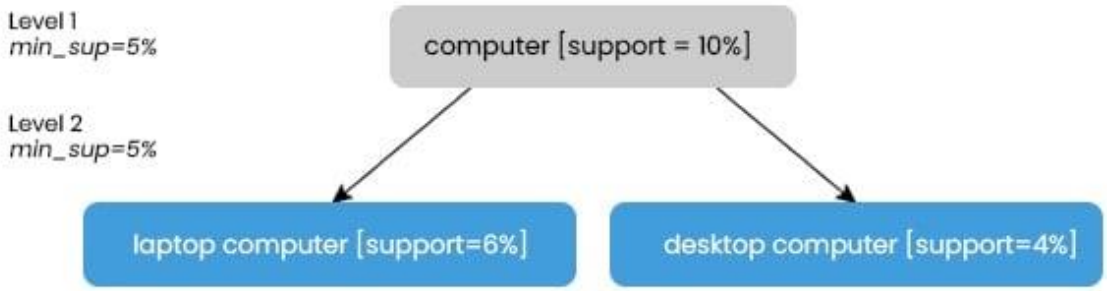
Finding engaging purchasing trends with such raw or basic data is challenging. Finding significant correlations with specific goods might be difficult if they occur in a tiny percentage of transactions, such as the "*Dell Studio XPS 16 Notebook*" or the "*Logitech VX*

Nano Cordless Laser Mouse." Due to low expected group buying, this bundle is unlikely to reach critical mass.

In contrast, we anticipate that it is simpler to discover robust correlations between broad categories of these things, such as "Dell Notebook" and "Cordless Mouse."

Multilevel Association Rule Mining

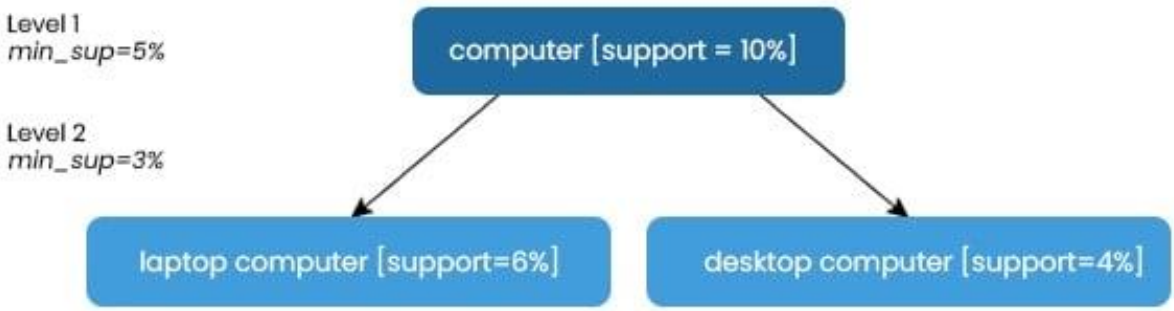
Many-level, or multilevel, association rules are a type of association rule developed via data mining at multiple abstraction levels. Rules of the association at several levels are mined effectively by employing idea hierarchies inside a support-confidence framework. Frequent item sets are often calculated using a top-down approach, where counts are amassed for each idea level from the most general to the most particular, and so on, until no more frequent item sets can be identified. For each tier, you can use Apriori or any other method for finding frequently occurring item sets.



Methods for Association Rule Mining:

What follows is a discussion of many alternatives to this method, each of which requires "playing" with the support threshold somewhat differently. Nodes represent things or item sets that have been investigated, and nodes with thick margins represent items or set examinations that have occurred often.

We have uniform support by providing the same amount of backing across the board.



- Minimal Support Criterion Method

Mining at each level of abstraction uses the same minimal support criterion.

A minimum support requirement of 5% is applied (for mining from "*computer*" down to "*laptop computer*"). The terms "*computer*" and "*laptop computer*" are standard, whereas "*desktop computer*" is rarely used.

Using a standard minimum support level can streamline the search process. The process is extremely straightforward because consumers need minimal support from only one. Given that every ancestor is a superset of its descendants, we may use an Apriori-like optimization strategy:

The search will ignore any item for which the ancestors do not have minimal support.

Despite its popularity, the uniform support method has its challenges. Products with a greater degree of abstraction are more likely to appear frequently than those with a lower level of abstraction. Because of this, it's essential to find a happy medium between setting the minimum support threshold high enough to catch all but the most vital links and making it too low to miss any significant ones. Too low of a point might cause the generation of numerous irrelevant linkages at very high abstraction levels.

- Reduced Support Method

This is the impetus for the following strategy. Through the use of lower levels of minimal support (hence referred to as "*reduced support*"):

A lower threshold of support is required at lower levels of abstraction. The threshold is lower at higher levels of abstraction. The above figure shows that 5% and 3% minimum support are required for levels 1 and 2, respectively. Computer, laptop computer, and desktop computer are all examples of common noun phrases here.

Building up user-specific, item- or group-based minimal support levels may be preferable when mining multilayer rules, as users or experts typically have insight into which groups are more significant than others. Set minimum support levels depending on the product price or items of interest; for instance, pay special attention to the association patterns, including those categories (such as "camera with price over $1000" or "Tablet PC") and the system will prioritize such patterns.

The threshold is often set to the lowest value among the participating groups for mining patterns containing elements from many groups with varying support thresholds. This way, useful patterns comprising elements from the least-supported group won't be filtered out. Meanwhile, it's important to maintain the minimum required level of support for each group to prevent the accumulation of boring collections of things from any group. After the itemset mining, different metrics of interestingness may be applied to draw out the most pertinent rules.

When mining under limited or group-based support, remember that the Apriori attribute does not always hold evenly across all objects. On the other hand, effective strategies may be created by expanding the property.

As a negative byproduct of the "*ancestor*" relationships among things, pattern mining multilevel association rules generates numerous duplicate rules across several abstraction

levels. Using the idea hierarchy, one can see that the "laptop computer" is the progenitor of "Dell laptop computer," prompting us to evaluate the following rules:

Clients who made purchases in AllElectronics transactions are represented by the variable X

Support = 8%, Confidence = 70% buys(X, "laptop computer") > buys(X, "HP printer").

compare('buys(X, 'Dell laptop computer') > compare('buys(X, 'HP printer')] [support = 2%, c onfidence = 72%] give something new or different?" We should eliminate the latter, less broa d rule if it doesn't add anything to the conversation. Let's consider some possible methods of determining this. If the items in R2 can be acquired by replacing them with their predecessors in a concept hierarchy, then R2 is an ancestor of R1.

The reason is that "*laptop computer*" predates "Dell laptop computer." This definition suggests that a rule is redundant if its support and confidence are near the "*anticipated*" values based on a rule's parent.

Multidimensional Associations Rule in Data Mining

Association rules with a single implying predicate, such as "*buys*," have been the focus of our attention so far. The Boolean association rule buys(X, "digital camera") buys(X, "HP printer"), for instance, may be found by mining our AllElectronics database.

Each unique predicate in a rule is referred to as a dimension, in keeping with the terminology used in multidimensional databases. Thus, the Rule is a single-dimensional or intra-dimensional association rule since it has a single distinct predicate (such as buys) with multiple occurrences (i.e., the predicate appears more than once within the rule). Rules like this may often be extracted from transactional data. You can learn more about six stages of data science processing to grasp the above topic better.

When analyzing sales and related information, it's not uncommon to combine it with relational data or include it in a data warehouse rather than relying just on transactional data. Multidimensionality is a hallmark of such data warehouses. For instance, a relational database may store additional information linked with sales transactions outside just the things purchased.

Associated with the goods and the deal itself, including the description of the goods or the location of the sale's branch. A customer's age, profession, credit score, income, and address are all examples of relational data that may maintain with what they bought. To mine multi-predicate association rules like age(X, "20-29"), occupation(X, "student") purchases(X, "laptop"), we must treat each database attribute or warehouse dimension as a predicate.

Multidimensional association rules are those that involve three or more dimensions or predicates. The rule has three unique predicates: age, occupation, and purchases. For this reason, we may confidently declare that it contains no recursive predicates. Interdimensional association rules span many dimensions but have no common predicates. Multi-dimensional association rules with repeated predicates can also be mined for helpful information. These rules have predicates that appear more than once. Hybrid-dimensional association rules are a term for these types of guidelines. For instance, consider the rule age(X, "20...29") > buys(X,

"laptop") > buys(X, "HP printer"), where each iteration of the predicate buys is applied to a different X.

Attributes in a database can be either nominal or quantitative. Nominal attributes (categorical attributes) have "*names of objects*" as their values. There is no hierarchy among the potential values for nominal properties (e.g., occupation, brand, color).Age, salary, and cost are all examples of quantitative qualities, implying a hierarchy of relative importance.

Multidimensional Association Rules in Data Mining Can be Divided Into Two Broad Camps Based on Quantitative Features:

- One method involves leveraging existing idea hierarchies to discretize quantitative features. There is a discretization step before mining. In the case of income, for instance, interval labels such as "0..20K," "21K..30K," "31K..40K," and so on can be used in place of the original numeric values of this attribute thanks to a concept hierarchy. The present case involves a fixed and specified discretization.The interval labels on the discretized numeric attributes allow them to be interpreted as nominal attributes (where each interval represents a category). Using a static discretization of quantitative characteristics, we refer to this as mining multidimensional association rules.
- The second method divides quantitative characteristics into discrete groups, or "*bins*," according to how the corresponding data is distributed. During the mining process, these containers may be mixed together again. For example, a dynamic discretization procedure has been devised to maximize the reliability of the rules mined. Association rules discovered using this method are sometimes known as (vibrant) quantitative association rules. Then numerical attribute values are treated as quantities rather than as established ranges or categories. Let's look into each of these methods for mining MDA rules.

So as not to get too complicated, we will only be talking about the laws of the relationship between dimensions. In multidimensional association rule mining, we look for frequent predicate sets rather than regular, frequent item sets, as in single-dimensional association rule mining. A set with k conjunctive predicates is called a k-predicate set in set theory. For example, Rule's set of predicates "*age*, occupation, *buys*" is a 3-predicate set.